

TECHCATALYST DE 2026

Git and GitHub Workflow

Turn work into intentional, reviewable, and recoverable snapshots.

Week 3 Git

Inspect

Diff

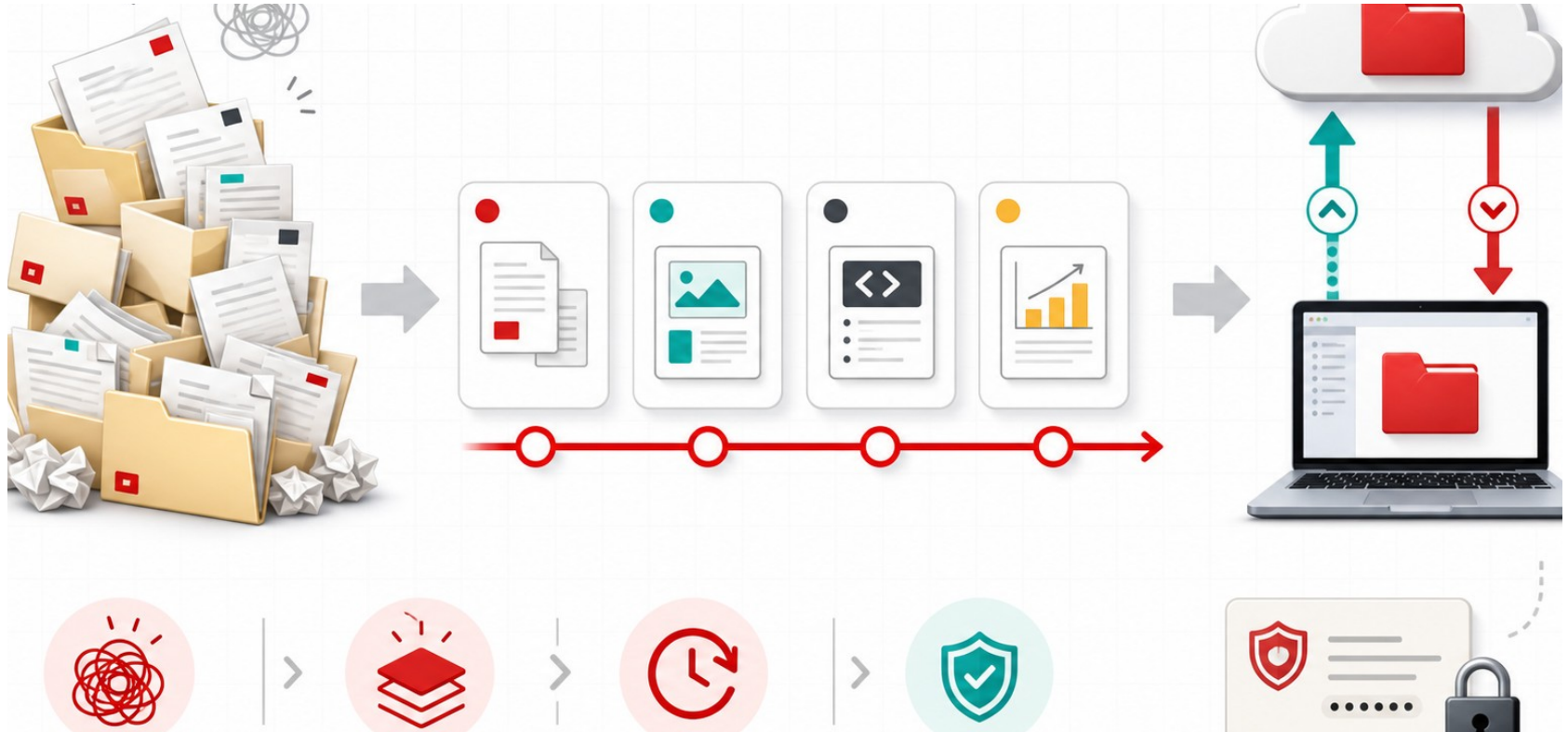
Stage

Commit

Pull

Push

Git replaces file-copy chaos with useful history



What you will be able to do

Learning goals	By the end, you can...
Distinguish	Explain how Git differs from GitHub
Model	Trace a change through working, staging, local, and remote areas
Inspect	Use status and diff before changing repository state
Stage	Select only the files that belong in the next snapshot
Commit	Write a concise message that explains one coherent change
Synchronize	Choose clone, fetch, pull, or push by intent
Protect	Keep environments, generated output, and secrets out of Git
Recover	Use small commits to make mistakes easier to understand

Weeks 1 to 4 are an AI-free zone

You cannot review code you could not have written.

What is allowed

- Write and debug your own code
- Use official docs and course references
- Ask peers to explain, not complete
- Call APIs and SDKs when the lesson requires them

Why we do this

- Fluency comes from typing and testing
- Error messages build debugging judgment
- Interviews still test fundamentals
- Later AI use should amplify skills you already own

01

Git workflow

Make every change intentional and recoverable.

Git and GitHub solve different problems

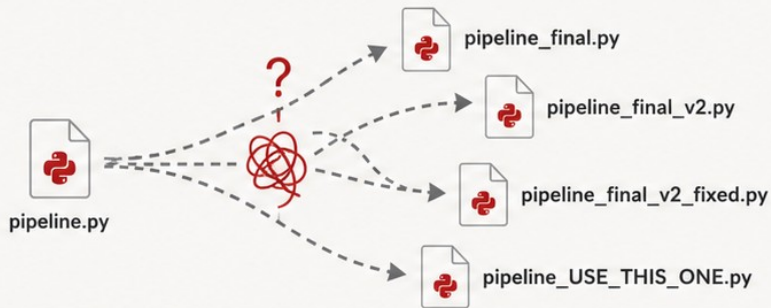
Tool	What it is	Where it works
Git	Version control for snapshots and branches	Your local project
GitHub	Remote hosting, review, and collaboration	The web and remote repository

First habit: when unsure, run `git status` and read what Git tells you.

Why Git?

Without Git

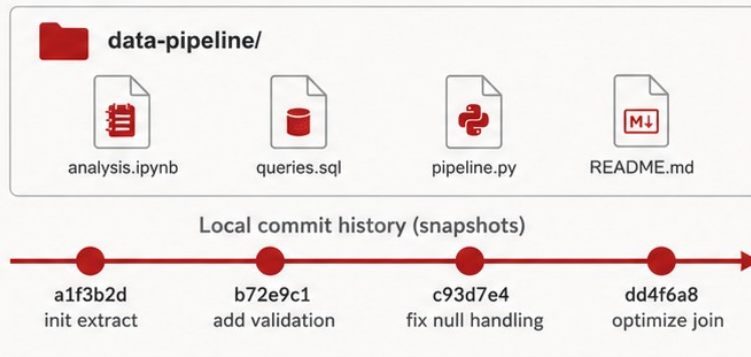
The “Save As” spiral — one project, many confusing copies



- ✗ Hard to know which file is latest
- ✗ No clean history of what changed
- ✗ Rollback means hunting through copies

With Git (local version control)

One project, clear history of snapshots



- ✓ One project folder
- ✓ Each commit is a named snapshot
- ✓ Compare or restore any earlier point
- ✓ Branch safely when experimenting

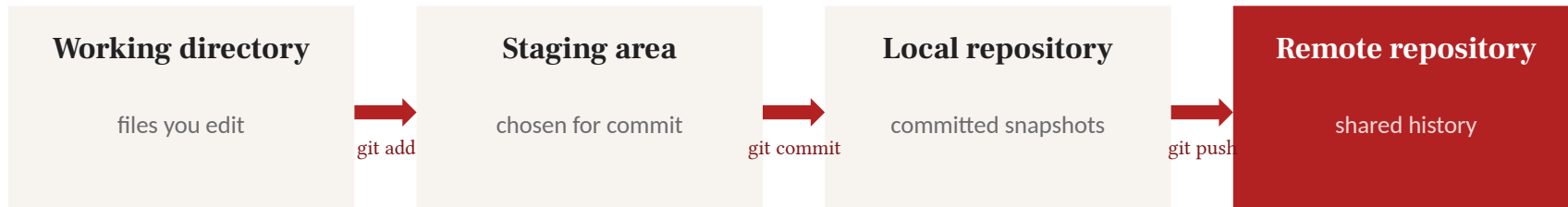


Git replaces messy duplicate filenames with a clear local history of snapshots.



Git works locally —
GitHub is optional.

The Git mental model has four areas



- A commit is a reviewed snapshot with a message
- `git status` tells you which area each change occupies
- Nothing moves to GitHub until you push

git status is your compass

```
cd student-work/week3/day3
```

```
git status
```

```
# after editing your notebook or script
```

```
git status --short
```

```
# inspect the working change
```

```
git diff
```

Git tracks a repository, not your whole computer. Confirm the project, read the status, and inspect the change before you stage it.

The edit, stage, commit loop

```
git status --short
# stage the intended file
git add medallion_etl.py
git diff --staged
git commit -m "Add silver cleaning rules"
git status --short
```

- Edit creates a working change
- Stage chooses the next snapshot
- Commit records the reviewed result
- Repeat in small, explainable steps

Review the diff before you commit

```
git diff          # unstaged
git add medallion_etl.py
git diff --staged  # proposed commit
git restore --staged notes.txt
```

`git diff --staged` is the pre-commit review habit that catches accidental files, debug output, and unrelated edits.

Good commits tell a useful story

Do

- Use the imperative: Add, Fix, Refactor
- Keep the first line concise
- Commit one coherent change
- Explain what and why

Avoid

- stuff, changes, fixed it, final
- Large mixed commits
- Generated environments or outputs
- Committing without reviewing the diff

Generated files and secrets do not belong in Git

```
# Python environments and caches
```

```
.venv/
```

```
__pycache__/
```

```
.ipynb_checkpoints/
```

```
# Secrets
```

```
.env
```

```
*.key
```

```
# Local outputs
```

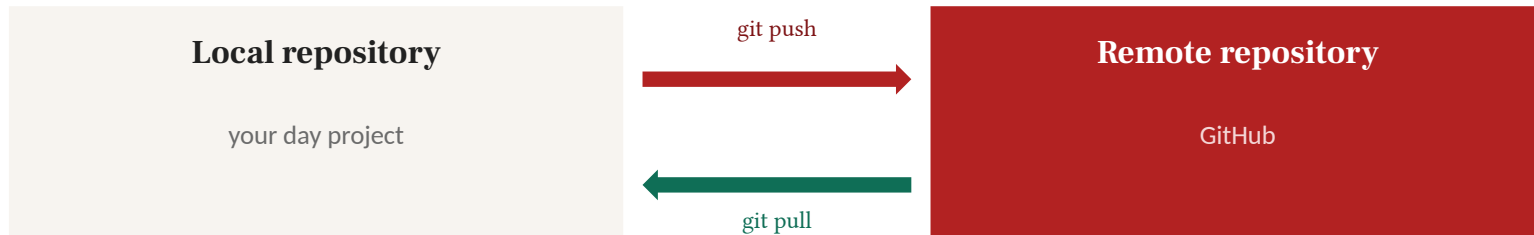
```
data/bronze/
```

```
data/silver/
```

The lesson: a committed credential is a security incident. Remove it from use immediately and rotate it.

The trap: .gitignore only affects untracked files. If a secret was already committed, ignoring it does not erase history.

Local and remote are separate copies



- Local work does not appear remotely until you push
- Remote work does not appear locally until you fetch or pull
- Pull before starting shared work, then push a clean history

Choose the remote command by intent

Command	What it does
git clone <url>	Copy a remote repository and connect origin
git fetch	Download remote history without merging
git pull	Fetch and integrate remote changes
git push	Send local commits to the remote

Build a useful three-commit story

Goal: turn one messy change into a reviewable sequence of snapshots.

25 min · pairs

- Inspect status and the working diff
- Separate unrelated changes
- Stage and review each proposed commit
- Write messages that explain what changed and why

Stretch: explain when fetch is safer than pull while reviewing remote work.

Deliverable: a three-commit history plus a partner explanation of the staging choices.

Git official references

Reference	Use it for
Pro Git book	Mental models, history, branching, and remotes
Git reference	Authoritative command behavior
GitHub Git guide	Remote collaboration workflow
GitHub secret scanning docs	Credential protection and response
gitignore documentation	Excluding generated and local files

Checked July 10, 2026. Prefer official documentation when older examples or remembered syntax disagree.

Useful history is built deliberately

1

Git tracks local snapshots. GitHub hosts and shares repositories.

2

`git status` is the first command when repository state is unclear.

3

The staging area lets you design the next commit.

4

Review `git diff --staged` before every commit.

5

Secrets and generated environments never belong in repository history.

Exit ticket: What evidence should you inspect before pressing commit?